

AGEN 6080 Homework 1

Problem 1

a)

$$U = \frac{\eta}{r} \left[1 + \sum_{\ell=2}^{\infty} \sum_{m=0}^{\ell} \left(\frac{R_i}{r} \right)^{\ell} P_{\ell,m} [\sin(\phi)] C_{\ell,m} \cos(m\lambda) + S_{\ell,m} \sin(m\lambda) \right]$$

Legendre Polynomial Degree 2 and 3:

3: $\frac{1}{2}(5x^3 - 3x)$

2: $\frac{1}{2}(3x^2 - 1)$

1: x

0: 1

$$\rightarrow U = \frac{\eta}{r} \left[1 + \left(\frac{R_E}{r} \right)^2 P_{2,0} \sin \phi (C_{2,0}) + \left(\frac{R_E}{r} \right)^3 P_{3,0} \sin \phi (C_{3,0}) \right]$$

By Def $C_i = -J_i$

$$U = \frac{\eta}{r} \left[1 - \left(\frac{R_E}{r} \right)^2 \frac{1}{2} (3 \sin^2 \phi - 1) - \left(\frac{R_E}{r} \right)^3 \frac{1}{2} (5 \sin^3 \phi - 3 \sin \phi) \right] \quad \text{For body frame}$$

where $r = \sqrt{x^2 + y^2 + z^2}$

$\sin \phi = \frac{z}{r}$

$$a = a_1 + a_2 + a_3$$

$$a = \nabla U = \begin{bmatrix} \partial U / \partial x \\ \partial U / \partial y \\ \partial U / \partial z \end{bmatrix} = \frac{\partial U}{\partial r} \hat{r} + \frac{1}{r} \frac{\partial U}{\partial \phi} \hat{\phi} \rightarrow$$

$$\hat{r} = \begin{bmatrix} \cos\phi \cos\lambda \\ \cos\phi \sin\lambda \\ \sin\phi \end{bmatrix} \quad \hat{\phi} = \begin{bmatrix} -\sin\phi \cos\lambda \\ -\sin\phi \sin\lambda \\ \cos\phi \end{bmatrix}$$

$$\begin{aligned} \frac{\partial U}{\partial r} &= -\frac{1}{r^2} - 4R_E^2 \cdot \frac{-3}{r^4} \int_2 \frac{1}{2} (3\sin^2\phi - 1) - 4R_E^3 \cdot \frac{-4}{r^5} \int_3 \frac{1}{2} (5\sin^2\phi - 3\sin\phi) \\ &= -\frac{1}{r^2} \left[-1 + R_E^2 \cdot \frac{3}{2r^2} \int_2 (3\sin^2\phi - 1) - R_E^3 \cdot \frac{2}{r^3} \int_3 (5\sin^2\phi - 3\sin\phi) \right] \end{aligned}$$

$$\frac{\partial U}{\partial \phi} = -\frac{4}{r^3} R_E^2 \int_2 \frac{1}{2} (6\sin\phi \cos\phi) - \frac{4}{r^4} R_E^3 \int_3 \frac{1}{2} (15\sin^2\phi \cos\phi - 3\cos\phi)$$

again $a = a_r \hat{r} + a_\phi \hat{\phi}$ $-\nabla U = a = -\frac{\partial U}{\partial r} \hat{r} - \frac{\partial U}{\partial \phi} \frac{1}{r} \hat{\phi}$ Def of gradient in spherical coords

$$\hat{r} = \frac{1}{r} \langle x, y, z \rangle \rightarrow \frac{x}{\sqrt{x^2+y^2+z^2}}, \dots$$

$$\hat{\phi} = \frac{1}{r\sqrt{x^2+y^2}} \langle -x^2, -y^2, \sqrt{x^2+y^2} \rangle$$

$$\text{So } a = \frac{a_r}{r} \begin{bmatrix} x \\ y \\ z \end{bmatrix} + \frac{a_\phi}{r\sqrt{x^2+y^2}} \begin{bmatrix} -x^2 \\ -y^2 \\ \sqrt{x^2+y^2} \end{bmatrix}$$

$$a_j = \frac{1}{r^3} \hat{r}$$

$$a_{j2} = a_{rj2} \hat{r} + a_{\phi j2} \hat{\phi} = \frac{3 \cdot 4 \int_2 R_E^2}{2 r^4} (3\sin\phi - 1) \hat{r} + \frac{3}{r^4} 4 \int_2 R_E^2 \sin\phi \cos\phi \hat{\phi}$$

$$a_{j3} = a_{rj3} \hat{r} + a_{\phi j3} \hat{\phi} = \frac{2}{r^5} 4 \int_3 R_E^3 (5\sin^2\phi - 3\sin\phi) \hat{r} - \frac{3 \cdot 4}{2r^5} 4 \int_3 R_E^3 (\cos^2\phi - 1) \cos\phi \hat{\phi}$$

Combine and simplify

$$\vec{a}_{BCJ} = -\frac{4}{r^3} \hat{r} + \frac{3}{2} \frac{4}{r^5} \int_2 R_E^2 \begin{bmatrix} x(5\frac{z^2}{r^2} - 1) \\ y(5\frac{z^2}{r^2} - 1) \\ z(5\frac{z^2}{r^2} - 3) \end{bmatrix} + \frac{4 \int_3 R_E^3}{2r^4} \begin{bmatrix} 5xz(7z^2 - 3r^2) \\ 5yz(7z^2 - 3r^2) \\ 35z^4 - 30z^2r^2 + 3r^4 \end{bmatrix}$$

ASBN 6070 Homework 1 cont

Problem 1 out

b) $\frac{dq}{dt} = 0$ for purely gravitational forces

Position Jacobian: $\frac{dq}{dr} = \begin{bmatrix} \frac{\partial a_x}{\partial x} & \frac{\partial a_x}{\partial y} & \frac{\partial a_x}{\partial z} \\ \frac{\partial a_y}{\partial x} & \frac{\partial a_y}{\partial y} & \frac{\partial a_y}{\partial z} \\ \frac{\partial a_z}{\partial x} & \frac{\partial a_z}{\partial y} & \frac{\partial a_z}{\partial z} \end{bmatrix} = \frac{\partial a_0}{\partial r} + \frac{\partial a_{jz}}{\partial r} + \frac{\partial a_{jz}}{\partial r}$

Identity: $\nabla(r^n) = nr^{n-2}\vec{r}$ and for Vector-Scalar product rule

$\frac{\partial a_0}{\partial r} = \frac{1}{r^3}(-4\vec{r}) = -4\left(\frac{1}{r^3}\vec{I} - \frac{3}{r^5}\vec{r}\vec{r}^T\right)$
 $-3r^{-5}\vec{r}$

$\frac{\partial f\vec{r}}{\partial r} = f\vec{I} + r(\nabla f)^T$
 (Thanks stuck exchange)

f: Scalar
 r: Vector

$\frac{\partial B}{\partial x} = \frac{3}{2r^7} \begin{bmatrix} x(r^2 - 5z^2) \\ y(r^2 - 5z^2) \\ z(3r^2 - 5z^2) \end{bmatrix}$

$\frac{\partial B}{\partial x} = \begin{matrix} r^2 - 5z^2 + 2x^2 \\ 2xy \\ 6xz \end{matrix}$

$\nabla\left(\frac{1}{r^3}\right) = -\frac{7}{r^5}\vec{r}$

$\frac{\partial B}{\partial y} = \begin{matrix} 2xy \\ (r^2 - 5z^2) + 2y^2 \\ 6yz \end{matrix}$

$\frac{\partial B}{\partial z} = \begin{matrix} x(2z - 10z) - 8xz \\ y(2z - 10z) - 8yz \end{matrix}$

$3r^2 - 5z^2 + (6z - 10z)z = 3r^2 - 5z^2 - 4z^2$

From $\frac{\partial f(r)}{\partial r} = f' + r(\nabla f)$ use B as in $f(B)$

So $\frac{\partial f(B)}{\partial r} = f' \frac{\partial B}{\partial r} + B(\nabla f)^T$ Ignore constants for now and $f = \frac{1}{r^3}$

$$\frac{\partial a_{j_2}}{\partial r} = \frac{3 \gamma_j R^2}{2r^7} \left[\begin{bmatrix} r^2 - 5z^2 + 2x^2 & 2xy & -8xz \\ 2xy & r^2 - 5z^2 + 2y^2 & -8yz \\ 8xz & 6yz & 3r^2 - 5z^2 - 4z^2 \end{bmatrix} + \begin{bmatrix} x(r^2 - 5z^2) \\ y(r^2 - 5z^2) \\ z(3r^2 - 5z^2) \end{bmatrix} \frac{-7}{r^4} \vec{r}^T \right]$$

$\nabla(r^{-9}) = -9r^{-10} \vec{r}$

$\frac{\partial f(C)}{\partial r} = f' \frac{\partial C}{\partial r} + B(\nabla f)^T$ $f = \frac{1}{r^9}$

$$a_{j_3} = \frac{1 \gamma_j R^3}{2r^9} \begin{bmatrix} 5xz(7z^2 - 3r^2) \\ 5yz(7z^2 - 3r^2) \\ 35z^4 - 30z^2r^2 + 3r^4 \end{bmatrix}$$

C

$$\begin{aligned} \frac{\partial C}{\partial x} &\Rightarrow 5z(7z^2 - 3r^2) + 5xz(-6x) \\ &= 5z(7z^2 - 3r^2) - 30x^2z \\ &= -30x^2z \\ &= -60z^2x + 12r^2x \end{aligned}$$

$3(x^2 + y^2 + z^2)^4 \Rightarrow (x^2 + y^2 + z^2)^2 \cdot x^4 + x^2y^2 + x^2z^2 + y^4 + y^2z^2 + z^4 + x^2y^2 + z^2x^2$
 $4x^3 + 4xy^2 + 4xz^2 \Rightarrow 12xr^2$

$$\begin{aligned} \frac{\partial C}{\partial y} &= -30xy^2z \\ &5z(7z^2 - 3r^2) + 5yz(-6y) = 5z(7z^2 - 3r^2) - 30y^2z \\ &12y(r^2 - 5z^2) \end{aligned}$$

$$\begin{aligned} \frac{\partial C}{\partial z} &= 5x(13z^2 - 3r^2) \leftarrow 5x(7z^2 - 3r^2) + 5xz(14z - 6z) = 5x(7z^2 - 3r^2 + 8z^2) \\ &5y(13z^2 - 3r^2) \\ &16(5z^2 - 3r^2) \end{aligned}$$

ASBN 6030 Homework 1 cont

Prbhn 1 cont
b cont)

$$a_{ij} = \frac{4\mu_3 R_E^3}{2r^4} \left[\begin{array}{cc} 5z(7z^2 - 3r^2) - 30x^2z & -30xyz \\ -30xyz & 5z(7z^2 - 3r^2) - 30y^2z \\ 12x(r^2 - 5z^2) & 12y(r^2 - 5z^2) \end{array} \right] + \left[\begin{array}{c} 15x(5z^2 - r^2) \\ 15y(5z^2 - r^2) \\ 16z(5z^2 - 3r^2) \end{array} \right] + \left[\begin{array}{c} 5xz(7z^2 - 3r^2) \\ 5yz(7z^2 - 3r^2) \\ 35z^4 - 30z^2r^2 + 3r^4 \end{array} \right] \frac{-9}{r^4} \hat{r}$$



de b anti

$$\frac{\partial \vec{a}}{\partial \eta} = \frac{1}{\eta} \vec{a}$$

$$\frac{\partial \vec{a}}{\partial J_2} = \frac{3}{2} \frac{1}{r^7} R_E^2 \begin{bmatrix} x(r^2 - 5z^2) \\ y(r^2 - 5z^2) \\ z(3r^2 - 5z^2) \end{bmatrix}$$

$$\frac{\partial \vec{a}}{\partial J_3} = \frac{1}{2} \frac{1}{r^9} R_E^3 \begin{bmatrix} 5xy(7z^2 - 3r^2) \\ 5yz(7z^2 - 3r^2) \\ 3r^4 - 30r^2z^2 + 35z^4 \end{bmatrix}$$

Comble: $\frac{\partial \vec{a}}{\partial \vec{r}} = \frac{\partial \vec{a}_0}{\partial \vec{r}} + \frac{\partial \vec{a}_{J_2}}{\partial \vec{r}} + \frac{\partial \vec{a}_{J_3}}{\partial \vec{r}}$

$$= -\frac{1}{r^3} \mathbb{I}_{3 \times 3} - \frac{3}{r^5} \vec{r} \vec{r}^T$$

$$+ \frac{3}{2} \frac{J_2 R_E^2}{r^7} \begin{bmatrix} r^2 - 5z^2 + 2x^2 & 2xy & -8xy \\ 2xy & r^2 - 5z^2 + 2y^2 & -8yz \\ 6xz & 6yz & 3r^2 - 5z^2 - 3r^4 \end{bmatrix} + \begin{bmatrix} x(r^2 - 5z^2) \\ y(r^2 - 5z^2) \\ z(3r^2 - 5z^2) \end{bmatrix} \frac{-7}{r^5} \hat{r}^T$$

$$+ \frac{1}{2} \frac{J_3 R_E^3}{r^9} \begin{bmatrix} 5z(7z^2 - 3r^2) - 30x^2z & -30xyz & 15x(5z^2 - r^4) \\ -30xyz & 5z(7z^2 - 3r^2) - 30y^2z & 15y(5z^2 - r^4) \\ 12x(r^2 - 5z^2) & 12y(r^2 - 5z^2) & 35z^4 - 30z^2r^2 + 3r^4 \end{bmatrix} + \begin{bmatrix} 5xz(7z^2 - 3r^2) \\ 5yz(7z^2 - 3r^2) \\ 35z^4 - 30z^2r^2 + 3r^4 \end{bmatrix} \frac{-9}{r^9} \hat{r}^T$$

Structure for Code?

$$X = \begin{matrix} 3 \times 3 & 3 \times 3 \\ r, V, \eta, J_2, J_3 \end{matrix} \rightarrow J_n = 9 \times 9$$

$$9 \times 9 = \frac{\partial a_x}{\partial x} \frac{\partial a_x}{\partial y} \frac{\partial a_x}{\partial z} \frac{\partial a_x}{\partial x} \frac{\partial a_x}{\partial y} \frac{\partial a_x}{\partial z} \frac{\partial a_x}{\partial \eta} \frac{\partial a_x}{\partial J_2} \frac{\partial a_x}{\partial J_3}$$

$$\frac{\partial a_y}{\partial x}$$

$$\frac{\partial a_z}{\partial x}$$

$$\frac{\partial a_x}{\partial x}$$

$$\vdots$$

compact

$$\dot{X} = A X \quad \text{Went (STM)} = \begin{bmatrix} \dot{r} \\ \dot{V} \\ \dot{\eta} \\ \vdots \end{bmatrix}$$

$$A = \begin{bmatrix} \frac{\partial \dot{r}}{\partial r} & \frac{\partial \dot{r}}{\partial V} & \frac{\partial \dot{r}}{\partial \eta} \\ \frac{\partial \dot{V}}{\partial r} & \frac{\partial \dot{V}}{\partial V} & \frac{\partial \dot{V}}{\partial \eta} \\ \frac{\partial \dot{\eta}}{\partial r} & \frac{\partial \dot{\eta}}{\partial V} & \frac{\partial \dot{\eta}}{\partial \eta} \end{bmatrix} \oplus \dots$$

ASEN 6080 Homework 1

Problem 1:

C:

```

This is the check (A_calc - A_ref):
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
-1.27553940e-05 -2.67028809e-05 7.15255737e-05 0.00000000e+00 0.00000000e+00 0.00000000e+00 -2.24113464e-05 -2.38651410e-09 7.05718994e-05
-2.86102295e-05 2.47955322e-05 -1.35421753e-04 0.00000000e+00 0.00000000e+00 0.00000000e+00 4.38690186e-05 4.65661287e-09 -1.22070312e-04
7.15255737e-05 -1.33514404e-04 -1.14440918e-05 0.00000000e+00 0.00000000e+00 0.00000000e+00 -5.91278076e-05 -2.04890966e-08 4.95910645e-05
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00

```

Problem 2:

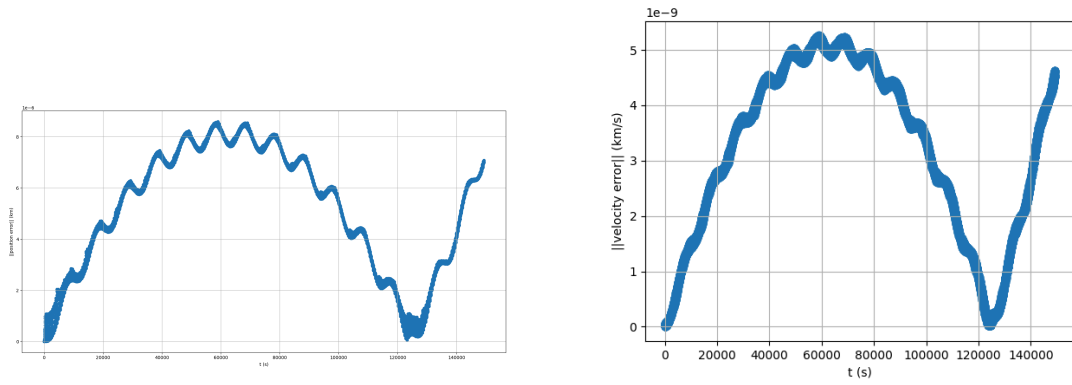
a:

```

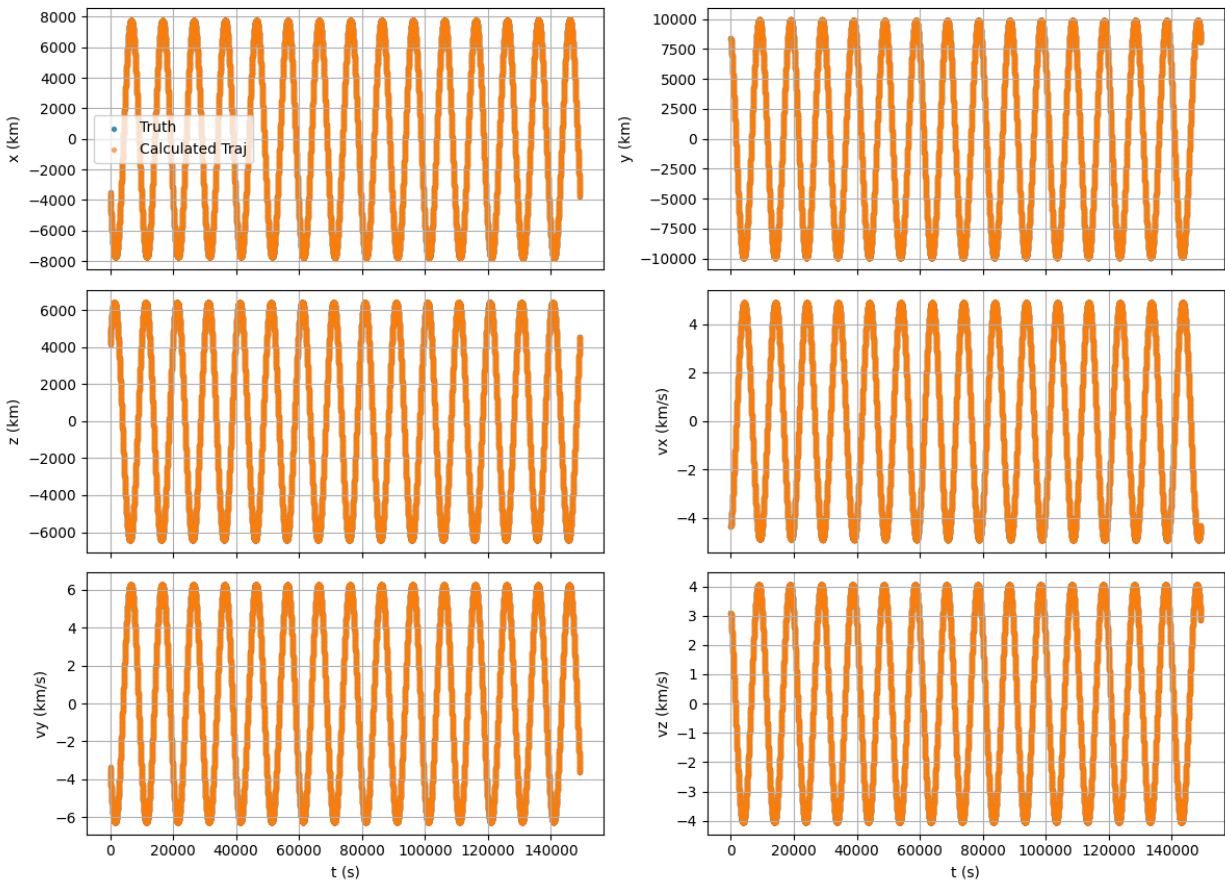
Max |pos error| (km): 8.590794758045835e-06
Max |vel error| (km/s): 5.262197764901428e-09

```

The Figures below show the position and velocity difference between my propagation and the given solution. The figure below the first two shows the states over time overlaid. The very small error indicated that my propagator is working properly. Since the error between the two is so small, the points also overlap almost perfectly, covering one set of data.



2a: Truth Trajectory vs Numerical Propagation

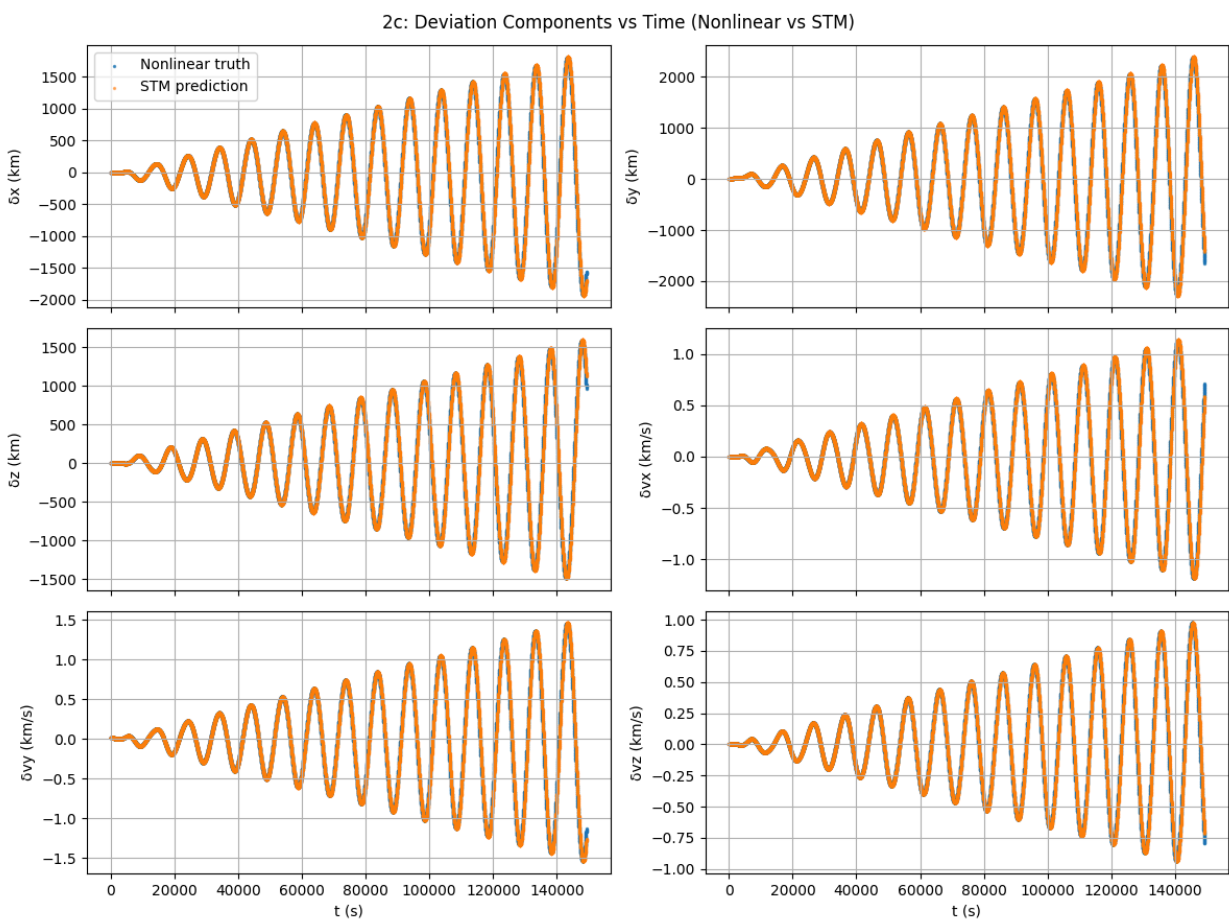


b)

```
Phi Difference  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00
0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00
0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00  0.0000000e+00
-8.74023438e-02 -8.44726562e-02 -1.00585938e-01  1.34765625e-01  9.27734375e-03 -1.40625000e-01  0.00000000e+00
1.70898438e-02  1.25000000e-01  1.56250000e-02 -4.49218750e-02  2.83203125e-02  7.03125000e-02 -1.85546875e-02
-1.56250000e-01 -4.88281250e-03  2.61230469e-02  2.00195312e-02  1.17187500e-02  4.51049805e-02  1.25000000e-01
0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
State Difference  0.0000000e+00
0.0000000e+00
0.0000000e+00
2.97851562e-02
-1.31835938e-02
-3.35693359e-03
0.00000000e+00
```

The difference between the supplied solutions and my function are less than a meter for all states, indicating the code works.

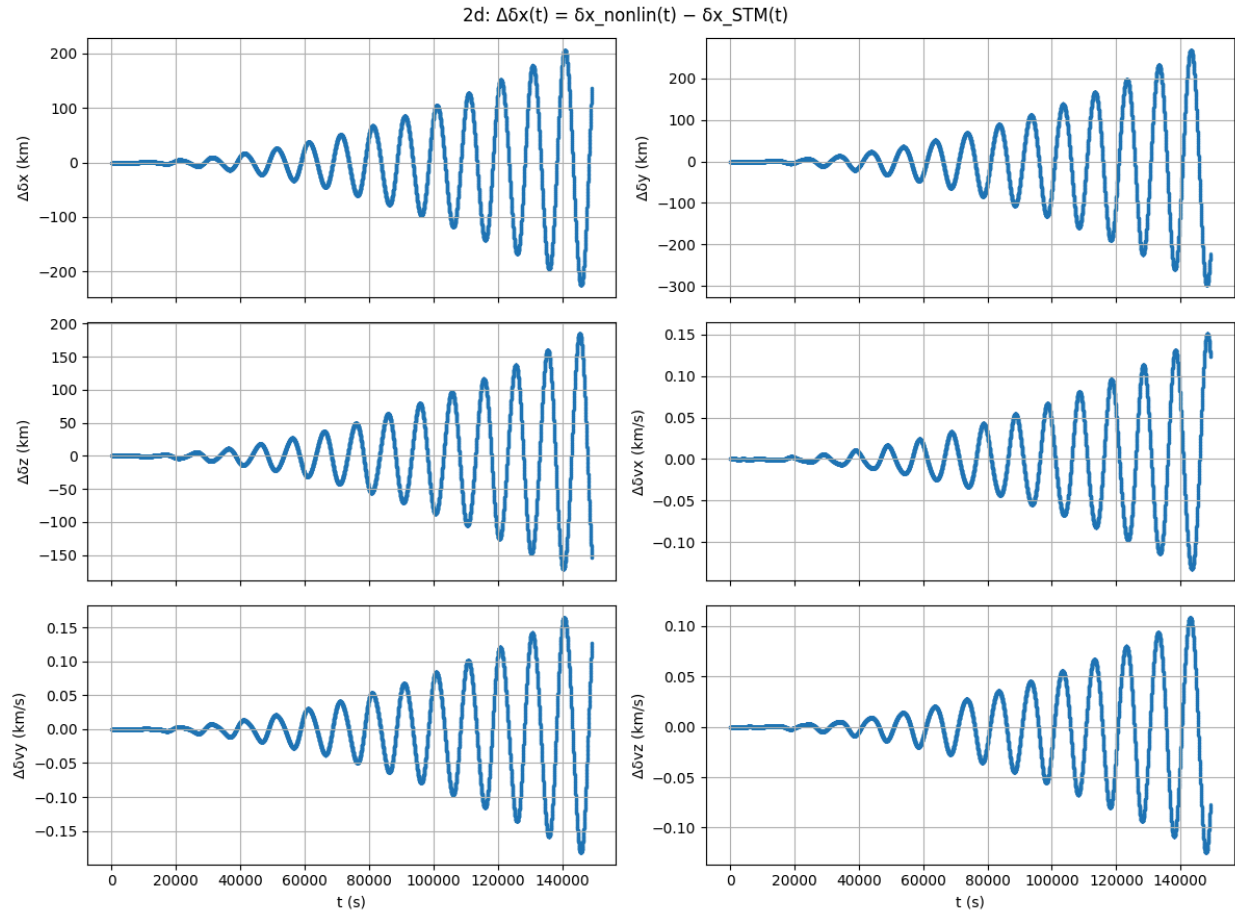
c:



The deviation histories produced by the two methods appear to be almost identical for the full 15 orbit propagation. As expected, the initial small perturbation causes the

trajectories to diverge from the unperturbed propagation. This divergence grows unbounded over time.

d:



The plots show that the STM's approximation of the error is relatively close to the true trajectory for roughly the first 20000 seconds. After that the deviations from the STM prediction grow radially unbounded. This divergence is expected because the STM provides only a first-order mapping and does not capture higher-order nonlinear effects that accumulate over long propagation times. This indicates that the STM approach is locally valid but as the nonlinear dynamics introduce more errors the deviation grows.

Problem 3:

a:

$$1) \rho = |\bar{R} - \bar{R}_s| \quad \dot{\rho} = \frac{(\bar{R} - \bar{R}_s) \cdot (\bar{V} - \bar{V}_s)}{\rho}$$

$$\hat{\rho} = \frac{\bar{r}}{\rho}$$

$$\rho = \sqrt{\bar{r}^T \bar{r}} \quad \text{so} \quad \boxed{\frac{\partial \rho}{\partial \bar{r}} = \frac{1}{2} (\bar{r}^T \bar{r})^{-1/2} \cdot 2 \bar{r}^T = \frac{\bar{r}^T}{\rho} = \hat{\rho}^T} \quad \left(\frac{\partial \rho}{\partial \bar{v}} = 0 \right)$$

derivative of $\bar{r}^T \bar{r} = 2 \bar{r}^T$

$$\dot{\rho} = \frac{\bar{R} \cdot \bar{V}}{\rho} \rightarrow \frac{\partial \dot{\rho}}{\partial \bar{R}_s} = -\frac{\Delta \bar{V}_s}{\rho} - \frac{\Delta \bar{R} \cdot \Delta \bar{V}}{\rho^2} \cdot \frac{\Delta \bar{R}_s}{\rho}$$

$$\frac{\partial \dot{\rho}}{\partial \bar{V}_s} = \frac{\Delta \bar{V}_s}{\rho} - \frac{\Delta \bar{R} \cdot \Delta \bar{V}}{\rho^2} \cdot \frac{\Delta \bar{R}_s}{\rho}$$

$$\frac{\partial \dot{\rho}}{\partial \bar{r}_s} = \frac{\Delta \bar{V}_s}{\rho} - \frac{\Delta \bar{R} \cdot \Delta \bar{V}}{\rho^2} \cdot \frac{\Delta \bar{R}_s}{\rho}$$

$$\boxed{\frac{\partial \dot{\rho}}{\partial \bar{R}} = \frac{\Delta \bar{V}^T}{\rho} - \frac{\dot{\rho}}{\rho^2} \Delta \bar{R}^T}$$

$$\boxed{\frac{\partial \dot{\rho}}{\partial \bar{V}} = \frac{\partial \bar{R} \cdot \bar{V}}{\partial \bar{V}} \cdot \frac{1}{\rho} = \frac{\Delta \bar{R}^T}{\rho} = \hat{\rho}^T}$$

$$c) \quad \frac{\partial \dot{\rho}}{\partial \bar{R}_s} = -\frac{\partial \dot{\rho}}{\partial \bar{R}} = -\hat{\rho}^T$$

$$\frac{\partial \dot{\rho}}{\partial \bar{V}_s} = 0$$

$$\frac{\partial \dot{\rho}}{\partial \bar{R}_s} = \left(\frac{\Delta \bar{V}^T}{\rho} - \frac{\dot{\rho}}{\rho^2} \Delta \bar{R}^T \right)$$

$$\frac{\partial \dot{\rho}}{\partial \bar{V}} = \hat{\rho}^T$$

not needed

b:

```
H_out1:
[[ 0.94372161  0.10817724 -0.31254953  0.          0.          0.          ]
 [-0.21643607  0.56357805 -0.45845237  0.94372161  0.10817724 -0.31254953]]

H_expected:
[[ 0.94372161  0.10817724 -0.31254953  0.          0.          0.          ]
 [-0.21643607  0.56357805 -0.45845237  0.94372161  0.10817724 -0.31254953]]

Max abs diff: 4.135580766728708e-15
```

The near zero difference between the provided values and the calculated ones indicate that the derivation was done correctly.

c:

The derivation for part c is under the derivation for a in the handwritten work.

The partials exist but we do not need them for this implementation. This is because the stations are on the surface of the Earth, rotating at the same speed as the ECEF frame. When doing the transport theorem to get the velocity of the station relative to the ECI frame, the first derivative term drops out, leaving just the rotation of the Earth across the original vector. We are assuming we know the rate at which the station move, not estimating it.

d:

```
H_out:
[[-0.94372161 -0.10817724  0.31254953]]

H_expected:
[[-0.94372161 -0.10817724  0.31254953]]

Max abs diff: 3.6637359812630166e-15
```

3D is also accurate to machine precision.

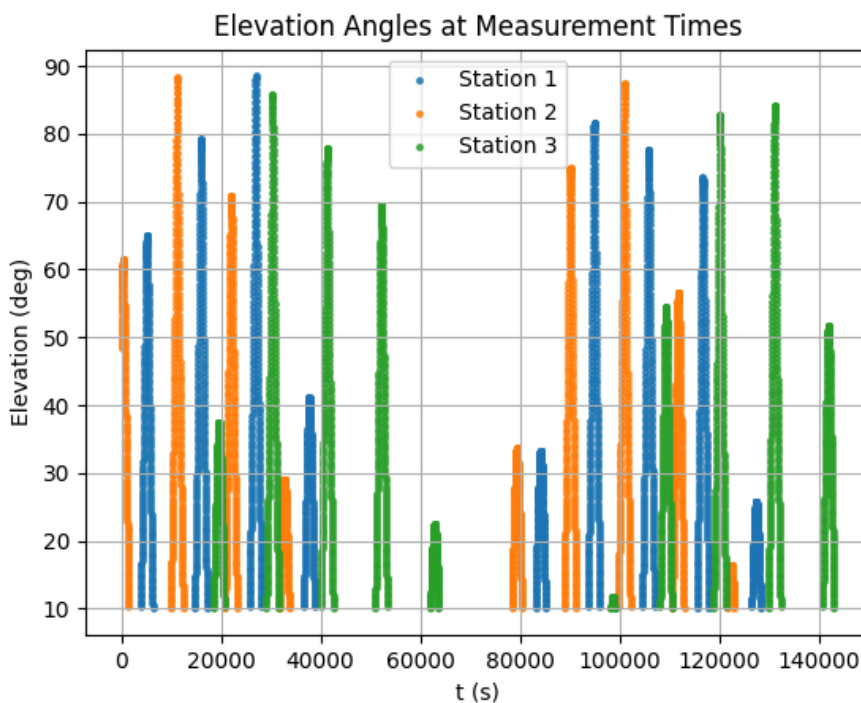
e:

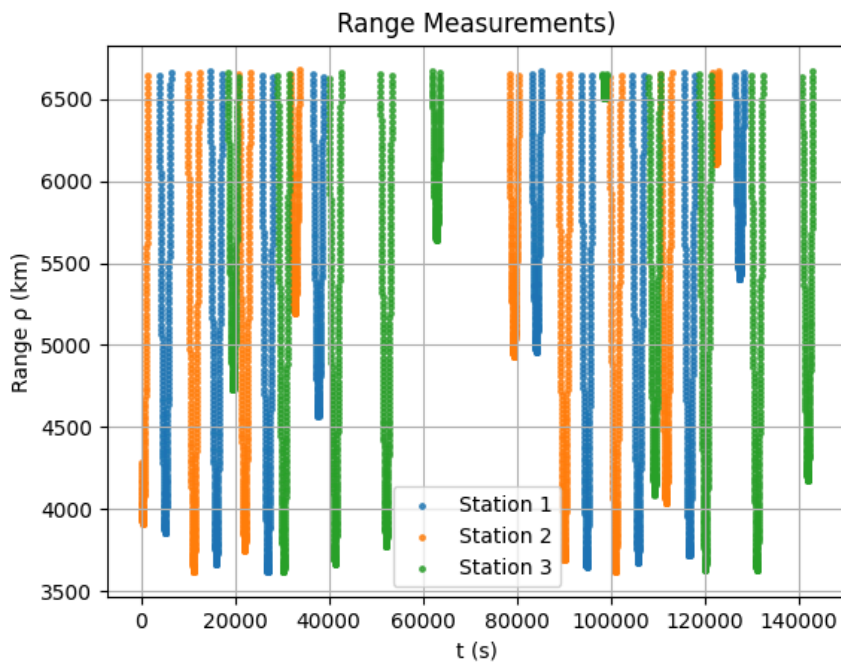
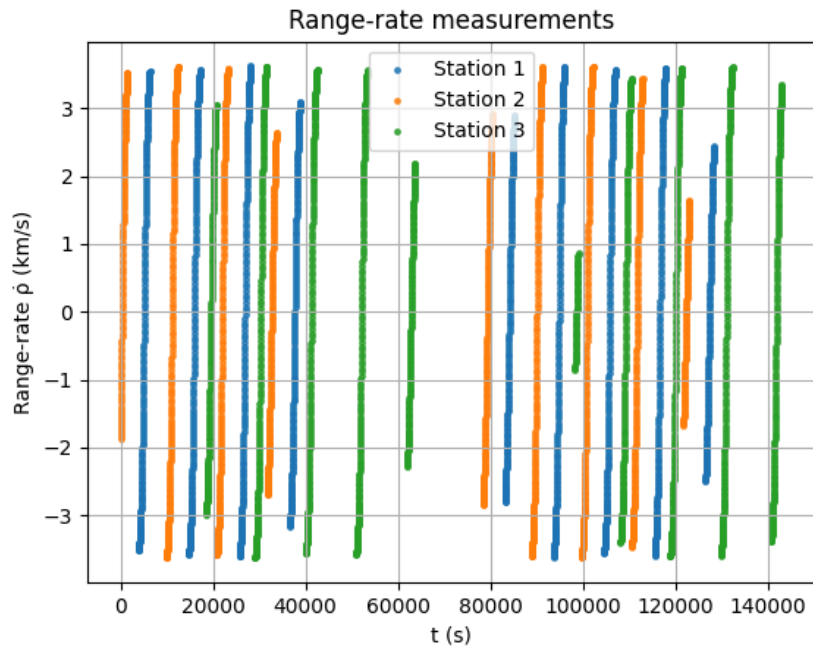
e) The bias will be a constant offset, so $\rho = \rho_{\text{Measured}} + \rho_{\text{off}}$. Assuming constant offset $\dot{\rho} = 0$ does not change. we will need partials WRT ρ_{off} , so the state is, now $\bar{x} = R, V, \rho_{\text{off}}$

so $H = \begin{bmatrix} \frac{\partial \rho}{\partial R} & \frac{\partial \rho}{\partial V} & \frac{\partial \rho}{\partial \rho_{\text{off}}} \\ \frac{\partial \dot{\rho}}{\partial R} & \frac{\partial \dot{\rho}}{\partial V} & \frac{\partial \dot{\rho}}{\partial \rho_{\text{off}}} \end{bmatrix}$

Problem 4:

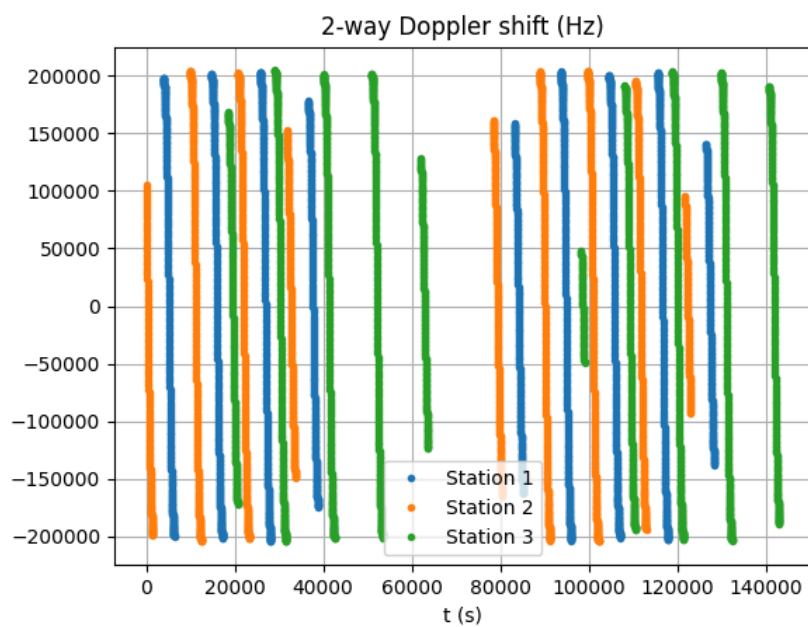
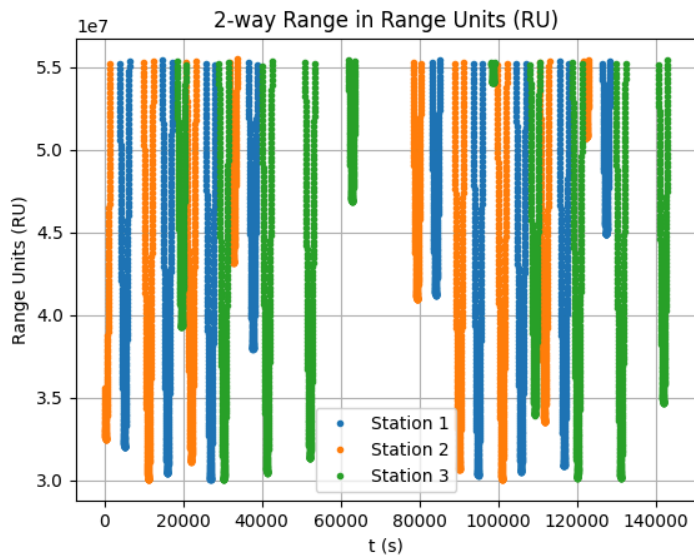
a and b:





The above figures show the range and range rate of the satellite relative to the 3 ground stations over the course of its 15 orbit period. The elevation graph shows that there are no measurements taken below 10 degree elevation, meaning the elevation mask works properly.

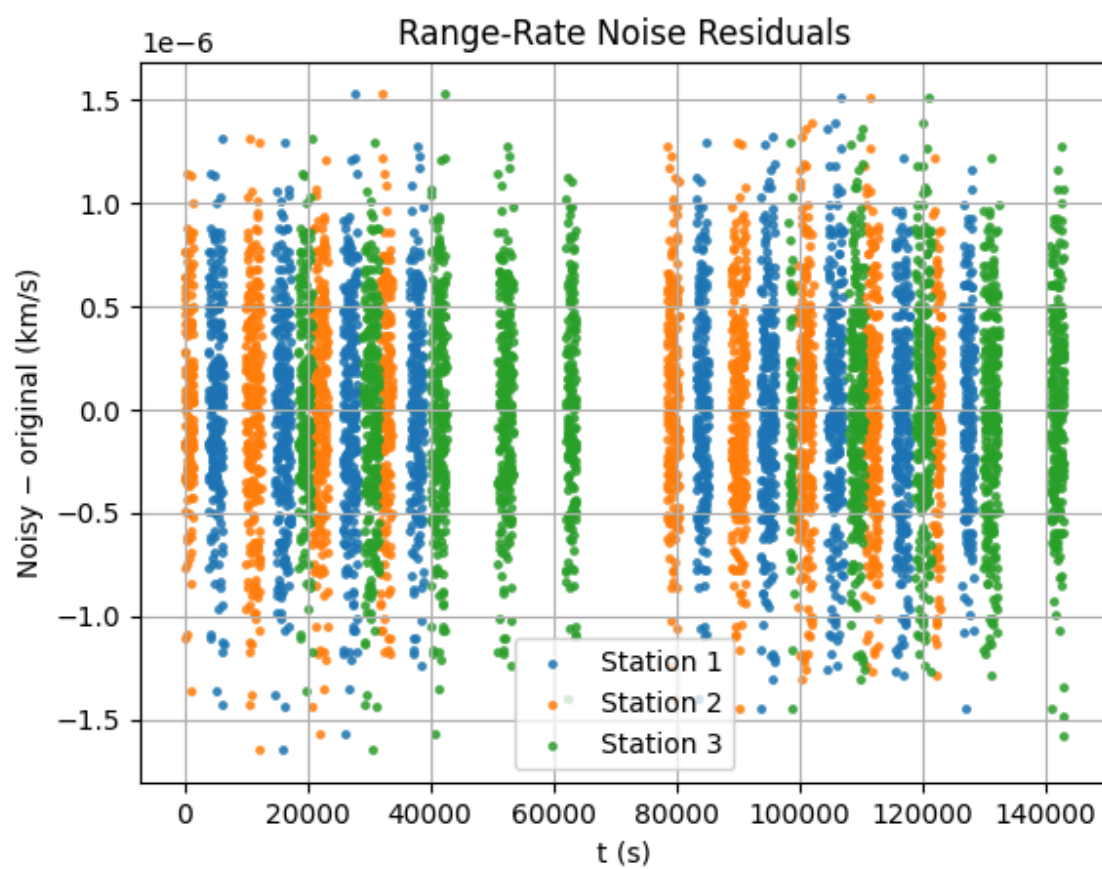
c:



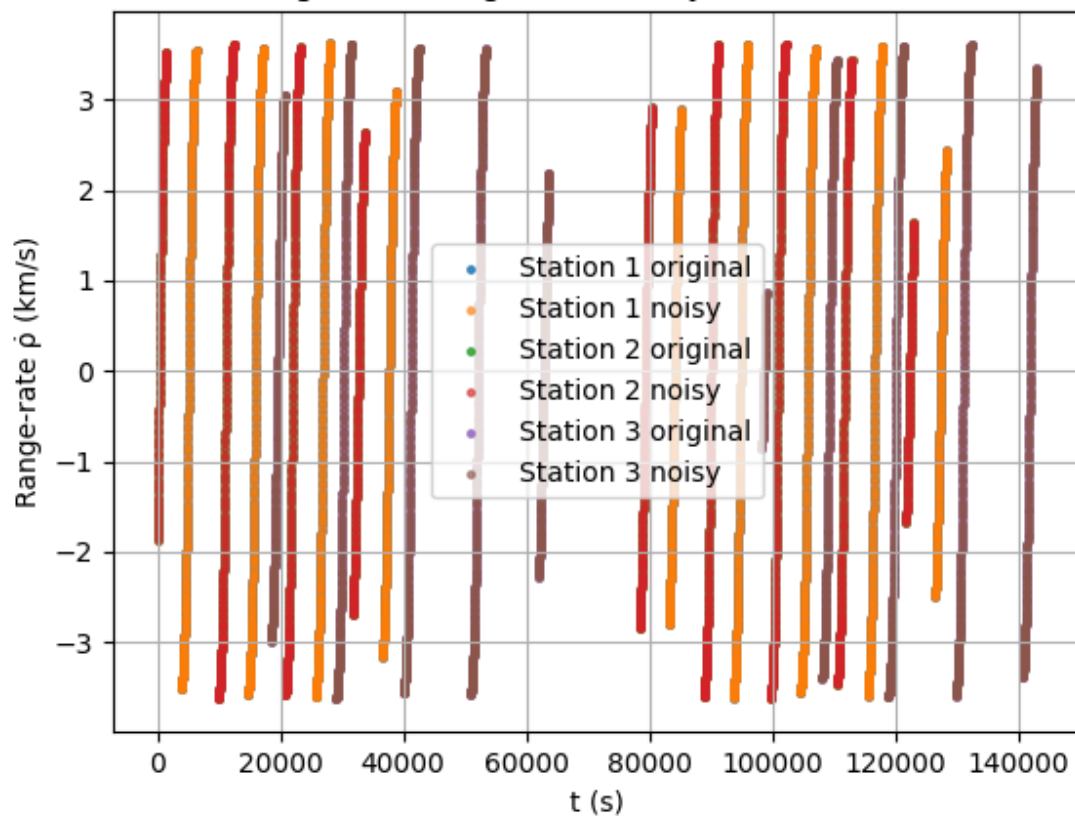
The figures above used the doppler method show that the data for the range looks identical to the ranging method used in part a. However, the two way doppler shift looks like a mirrored version of the range rate plot. This makes sense because regardless of the method, if they are both accurate the range should look the same. As for the doppler graph, the negative in front the frequency is the reason why the plot looks like a flipped version of the range rate.

d:

The figures below show the range rate with noise and the residuals of the range when noise is added. As expected, the position residual looks like random noise since we just added noise to our previously “perfect” measurements. However, the range rate plot looks identical to when there was no noise added. This may be problematic because it means that it could be easy to mistake noisy data for noise free data. Since the difference is so small it could lead to neglecting the fact that the data may not be completely accurate.



Range-rate: Original vs Noisy ($\sigma = 0.5$ mm/s)



Homework\Homework1\Homework_submission.py

```

1  import numpy as np
2  import pandas as pd
3  from scipy.integrate import solve_ivp
4  import json
5
6  ## I will make a git for next homework to make this all more legible
7
8  def accel_wJ2J3(r, mu, J2, J3, Re=6378, j2=True, j3=True):
9      """
10     Acceleration a(r) = a_mu + a_J2 + a_J3 in Cartesian
11     j2 j3 = togglable
12     """
13
14     x, y, z = r
15     r2 = x*x + y*y + z*z
16     rmag = np.sqrt(r2)
17
18     # Central term
19     a = -mu * r / (rmag**3)
20
21     # J2
22     if j2:
23         z2 = z*z
24         u2 = np.array([
25             x * (r2 - 5.0*z2),
26             y * (r2 - 5.0*z2),
27             z * (3.0*r2 - 5.0*z2)
28         ])
29         a -= (3.0*mu*J2*(Re**2) / (2.0*(rmag**7))) * u2
30
31     # J3
32     if j3:
33         z2 = z*z
34         r4 = r2*r2
35         z4 = z2*z2
36         C = 7.0*z2 - 3.0*r2
37         D = 3.0*r4 - 30.0*r2*z2 + 35.0*z4
38         w = np.array([
39             5.0*x*z*C,
40             5.0*y*z*C,
41             D
42         ])
43         a += (mu*J3*(Re**3) / (2.0*(rmag**9))) * w
44
45     return a
46
47
48 def dadr_wJ2J3(r, mu, J2, J3, Re=6378, j2=True, j3=True):

```

```

49     """
50     Gravity-gradient matrix G = da/dr (3x3) for a = a_mu + a_J2 + a_J3.
51
52     Returns
53     G : array, shape (3,3)
54     """
55     x, y, z = r
56     r2 = x*x + y*y + z*z
57     rmag = np.sqrt(r2)
58     I = np.eye(3)
59     rrT = np.outer(r, r)
60
61     # Central term
62     G = (mu / (rmag**5)) * (3.0*rrT - r2*I)
63
64     # J2
65     if j2:
66         z2 = z*z
67         u2 = np.array([
68             x * (r2 - 5.0*z2),
69             y * (r2 - 5.0*z2),
70             z * (3.0*r2 - 5.0*z2)
71         ])
72
73
74         M2 = np.array([
75             [(r2 - 5.0*z2) + 2.0*x*x, 2.0*x*y, -8.0*x*z],
76             [2.0*x*y, (r2 - 5.0*z2) + 2.0*y*y, -8.0*y*z],
77             [6.0*x*z, 6.0*y*z, (3.0*r2 - 5.0*z2) - 4.0*z2]
78         ])
79
80         coeff = (3.0*mu*j2*(r2**2)) / (2.0*(rmag**7))
81         G -= coeff * (M2 - (7.0/r2)*np.outer(u2, r))
82
83     # J3
84     if j3:
85         z2 = z*z
86         r4 = r2*r2
87         z4 = z2*z2
88
89         C = 7.0*z2 - 3.0*r2
90         D = 3.0*r4 - 30.0*r2*z2 + 35.0*z4
91
92         w = np.array([
93             5.0*x*z*C,
94             5.0*y*z*C,
95             D
96         ])
97
98         M3 = np.array([

```



```

99         [5.0*z*(C - 6.0*x*x), -30.0*x*y*z, 5.0*x*(C + 8.0*z2)],
100         [-30.0*x*y*z, 5.0*z*(C - 6.0*y*y), 5.0*y*(C + 8.0*z2)],
101         [12.0*x*(r2 - 5.0*z2), 12.0*y*(r2 - 5.0*z2), 16.0*z*(5.0*z2 - 3.0*r2)]
102     ])
103
104     coeff = (mu*J3*(Re**3)) / (2.0*(rmag**9))
105     G += coeff * (M3 - (9.0/r2)*np.outer(w, r))
106
107     return G
108
109
110 def da_dparams_wJ2J3(r, mu, J2, J3, Re=6378, j2=True, j3=True):
111     """
112     Parameter partials: da WRT mu, j2, j3.
113     """
114     a = accel_wJ2J3(r, mu, J2, J3, Re=Re, j2=j2, j3=j3)
115
116     da_dmu = a / mu
117
118     # da/dj2
119     if j2:
120         x, y, z = r
121         r2 = x*x + y*y + z*z
122         rmag = np.sqrt(r2)
123         z2 = z*z
124         u2 = np.array([
125             x * (r2 - 5.0*z2),
126             y * (r2 - 5.0*z2),
127             z * (3.0*r2 - 5.0*z2)
128         ])
129         da_dJ2 = -(3.0*mu*(Re**2) / (2.0*(rmag**7))) * u2
130     else:
131         da_dJ2 = np.zeros(3)
132
133     # da/dj3
134     if j3:
135         x, y, z = r
136         r2 = x*x + y*y + z*z
137         rmag = np.sqrt(r2)
138         z2 = z*z
139         r4 = r2*r2
140         z4 = z2*z2
141         C = 7.0*z2 - 3.0*r2
142         D = 3.0*r4 - 30.0*r2*z2 + 35.0*z4
143         w = np.array([5.0*x*z*C, 5.0*y*z*C, D])
144         da_dJ3 = (mu*(Re**3) / (2.0*(rmag**9))) * w
145     else:
146         da_dJ3 = np.zeros(3)
147
148     return da_dmu, da_dJ2, da_dJ3

```

```

149
150
151 def state_builder(state9, Re=6378, j2=True, j3=True):
152     """
153     Build the 9x9 A-matrix for augmented state:
154     X = [r(3), v(3), mu, J2, J3].
155
156     Returns
157     -----
158     A : 9x9 array of STM
159     """
160     state9 = np.asarray(state9, dtype=float).reshape(-1)
161
162     r = state9[0:3]
163     v = state9[3:6] # not needed for Jacobian
164     mu = state9[6]
165     J2 = state9[7]
166     J3 = state9[8]
167
168     G = dadr_wJ2J3(r, mu, J2, J3, Re=Re, j2=j2, j3=j3) # 3x3
169     da_dmu, da_dJ2, da_dJ3 = da_dparams_wJ2J3(r, mu, J2, J3, Re=Re, j2=j2, j3=j3)
170
171     A = np.zeros((9, 9), dtype=float)
172
173     # dr/dt = v
174     A[0:3, 3:6] = np.eye(3)
175
176     # dv/dt = a
177     A[3:6, 0:3] = G
178     # dv/dt partials wrt params
179     A[3:6, 6] = da_dmu
180     A[3:6, 7] = da_dJ2
181     A[3:6, 8] = da_dJ3
182     return A
183
184
185 ## Validation...please work
186
187 with open('Solution_Checks/prob1c_solution.json', "r") as validation_file:
188     my_json_obj = json.load(validation_file)
189
190     r = np.array(my_json_obj["inputs"]["state"]["r"], dtype=float)
191     v = np.array(my_json_obj["inputs"]["state"]["v"], dtype=float)
192     mu = float(my_json_obj["inputs"]["state"]["mu"])
193     J2 = float(my_json_obj["inputs"]["state"]["J2"])
194     J3 = float(my_json_obj["inputs"]["state"]["J3"])
195
196 state9 = np.hstack((r, v, mu, J2, J3))
197
198 A_calc = state_builder(state9, Re=6378, j2=True, j3=True)

```

```
199
200 # print(A_calc)
201
202 A_ref = np.array(my_json_obj["outputs"]["A_matrix"]["values"], dtype=float)
203 # print(A_ref)
204
205 check = A_calc - A_ref
206
207 df_check = pd.DataFrame(check)
208 print("This is the check (A_calc - A_ref):")
209 print(df_check.to_string(index=False, header=False,
210                           float_format=lambda x: f"{x: .8e}"))
211
212
213
214
215
216 ### 2a
217
218
219 import numpy as np
220 import sys
221 import json
222 import scipy as sp
223 import pandas as pd
224 import matplotlib.pyplot as plt
225
226 sys.path.append("../..")
227
228 from src.Functions.jacobians import *
229 from src.Functions.kep2cart import Keplarian_to_Cartesian
230
231 with open('prob2_OE.json', "r") as validation_file:
232     my_json_obj = json.load(validation_file)
233
234     a = np.float64(my_json_obj["orbit_elements"]["a_km"])
235     e = np.float64(my_json_obj["orbit_elements"]["e"])
236     i_deg = np.float64(my_json_obj["orbit_elements"]["i_deg"])
237     raan_deg = np.float64(my_json_obj["orbit_elements"]["raan_deg"])
238     w_deg = np.float64(my_json_obj["orbit_elements"]["argp_deg"])
239     ta_deg = np.float64(my_json_obj["orbit_elements"]["ta_deg"])
240
241 mu: np.float64 = 398600.4415
242 Re: np.integer = 6378
243 J2: np.float64 = 0.0010826269
244 tf: np.float64 = 15 * 2 * np.pi * np.sqrt((a**3/mu))
245
246 r_N, v_N, _ = Keplarian_to_Cartesian(mu, a, e, i_deg, raan_deg, w_deg, ta_deg)
247
248 X0 = np.array(np.hstack((r_N, v_N, mu, J2, 0)))
```

```
249
250 ##### Solution Check #####
251
252 truth = pd.read_csv(
253     "Solution_Checks/HW1_truth.txt",
254     sep=r"\s+",
255     header=None,
256     names=["t_s", "x_km", "y_km", "z_km", "vx_km_s", "vy_km_s", "vz_km_s"],
257     engine="python"
258 )
259
260 t_truth = truth["t_s"].to_numpy()
261 X_truth = truth[["x_km", "y_km", "z_km", "vx_km_s", "vy_km_s", "vz_km_s"]].to_numpy()
262
263
264
265 # Problem 2a
266 fun = lambda t, X: orbit_propagator_aug9(t, X, Re=Re, j2=True, j3=False)
267 sol = sp.integrate.solve_ivp(fun, (t_truth[0], t_truth[-1]), X0, t_eval=t_truth, rtol=1e-10,
268     atol=1e-10, method="DOP853")
269
270 X_model = sol.y[:6, :].T
271 cols = ["t_s", "x_km", "y_km", "z_km", "vx_km_s", "vy_km_s", "vz_km_s", "mu", "J2", "J3"]
272 data = np.column_stack((sol.t, sol.y.T))
273 df = pd.DataFrame(data, columns=cols)
274
275 # write CSV
276 df.to_csv("prob2a_traj.csv", index=False)
277 final_stm = sol.y[-1, :]
278
279
280 err = X_model - X_truth          # Nx6
281 pos_err = err[:, :3]             # Nx3
282 vel_err = err[:, 3:]             # Nx3
283
284 pos_err_norm = np.linalg.norm(pos_err, axis=1)
285 vel_err_norm = np.linalg.norm(vel_err, axis=1)
286
287 print("Max |pos error| (km):", pos_err_norm.max())
288 print("Max |vel error| (km/s):", vel_err_norm.max())
289
290
291
292 plt.figure()
293 plt.scatter(t_truth, pos_err_norm)
294 plt.xlabel("t (s)")
295 plt.ylabel("||position error|| (km)")
296 plt.grid(True)
297
```



```

298 plt.figure()
299 plt.scatter(t_truth, vel_err_norm)
300 plt.xlabel("t (s)")
301 plt.ylabel("||velocity error|| (km/s)")
302 plt.grid(True)
303
304 # plt.show()
305
306 # ----- 2a: Overlay truth vs model (6 subplots) -----
307
308
309 labels = ["x (km)", "y (km)", "z (km)", "vx (km/s)", "vy (km/s)", "vz (km/s)"]
310
311 fig, axs = plt.subplots(3, 2, figsize=(12, 9), sharex=True)
312 axs = axs.ravel()
313
314 for k in range(6):
315     axs[k].scatter(t_truth, X_truth[:, k], label="Truth", s=8, alpha=0.8)
316     axs[k].scatter(t_truth, X_model[:, k], label="Calculated Traj", s=8, alpha=0.6)
317     axs[k].set_ylabel(labels[k])
318     axs[k].grid(True)
319
320 axs[4].set_xlabel("t (s)")
321 axs[5].set_xlabel("t (s)")
322
323 axs[0].legend(loc="best")
324 fig.suptitle("2a: Truth Trajectory vs Numerical Propagation")
325 fig.tight_layout()
326 plt.show()
327
328
329
330 ##### 2b
331
332
333
334 import numpy as np
335 import sys
336 import json
337 import scipy as sp
338 import pandas as pd
339 sys.path.append("../..")
340 from src.Functions.jacobians import *
341 from src.Functions.kep2cart import Keplerian_to_Cartesian
342
343
344 ##### Problem 2b #####
345
346
347 mu: np.float64 = 398600.4415

```

```

348 Re: np.integer = 6378
349 J2: np.float64 = 1.08262668e-3
350
351 with open('Solution_Checks/prob2b_solution.json', "r") as validation_file2:
352     my_json_obj2 = json.load(validation_file2)
353
354     X0 = np.array(my_json_obj2["inputs"]["X0"]["values"], dtype=float)
355     Phi0 = np.array(my_json_obj2["inputs"]["Phi0"]["values"], dtype=float)
356     X_out = np.array(my_json_obj2["outputs"]["Xdot"]["values"], dtype=float)
357     phi_out = np.array(my_json_obj2["outputs"]["Phidot"]["values"], dtype=float)
358
359
360 t = 0
361 state9 = np.array([*X0[0:6], mu, X0[6], 0]) #tuple unpacking
362 remove = np.array([6, 8])
363
364 state_stuff = stm(t, state9, Phi0, remove, Re=6378, j2=True, j3=True)
365
366 state_check = state_stuff[0:7]
367 phi_check = state_stuff[9:]
368 phi_check_matrix_residuals = phi_check.reshape(7, 7) - phi_out
369 phi_check_matrix = phi_check.reshape(7, 7)
370
371 # print(state_check-X_out)
372
373 df_phi = pd.DataFrame(phi_check_matrix_residuals)
374 print("Phi Difference", df_phi.to_string(index=False, header=False, float_format=lambda x: f"
{x: .8e}"))
375
376 # df_phi = pd.DataFrame(phi_check_matrix)
377 # print(df_phi.to_string(index=False, header=False, float_format=lambda x: f"{x: .8e}"))
378
379 df_phi = pd.DataFrame(state_check-X_out)
380 print("State Difference", df_phi.to_string(index=False, header=False, float_format=lambda x:
f"{x: .8e}"))
381
382
383
384 ##### Problem 2c #####
385
386 import numpy as np
387 import sys
388 import json
389 import scipy as sp
390 import pandas as pd
391 import matplotlib.pyplot as plt
392 sys.path.append("../..")
393 from src.Functions.jacobians import *
394 from src.Functions.kep2cart import Keplarian_to_Cartesian
395

```

```

396
397 ##### Problem 2c #####
398
399
400 ##### Data Load In #####
401
402 with open('prob2_OE.json', "r") as validation_file:
403     my_json_obj = json.load(validation_file)
404
405     a = np.float64(my_json_obj["orbit_elements"]["a_km"])
406     e = np.float64(my_json_obj["orbit_elements"]["e"])
407     i_deg = np.float64(my_json_obj["orbit_elements"]["i_deg"])
408     raan_deg = np.float64(my_json_obj["orbit_elements"]["raan_deg"])
409     w_deg = np.float64(my_json_obj["orbit_elements"]["argp_deg"])
410     ta_deg = np.float64(my_json_obj["orbit_elements"]["ta_deg"])
411
412     x = np.float64(my_json_obj["pert"]["x"])
413     y = np.float64(my_json_obj["pert"]["y"])
414     z = np.float64(my_json_obj["pert"]["z"])
415     xdot = np.float64(my_json_obj["pert"]["dx"])
416     ydot = np.float64(my_json_obj["pert"]["dy"])
417     zdot = np.float64(my_json_obj["pert"]["dz"])
418
419
420 truth = pd.read_csv("prob2a_traj.csv")
421
422 ##### Problem #####
423
424 t_truth = truth["t_s"].to_numpy(dtype=float)
425
426 mu: np.float64 = 398600.4415
427 Re: np.integer = 6378
428 J2: np.float64 = 0.0010826269
429 # tf: np.float64 = 15 * 2 * np.pi * np.sqrt((a**3/mu))
430
431 r_N, v_N, _ = Keplarian_to_Cartesian(mu, a, e, i_deg, raan_deg, w_deg, ta_deg)
432 X0_pure = np.array(np.hstack((r_N, v_N, mu, J2, 0)))
433 X_pert = np.array(np.hstack((x, y, z, xdot, ydot, zdot, 0, 0, 0)))
434
435 X0 = X0_pure + X_pert
436 # Perturbed 2a
437
438 fun = lambda t, X: orbit_propagator_aug9(t, X, Re=Re, j2=True, j3=False)
439 sol = sp.integrate.solve_ivp(fun, (t_truth[0], t_truth[-1]), X0, t_eval=t_truth, rtol=1e-10,
440                               atol=1e-10, method="DOP853")
441
442 X_model = sol.y[:6, :].T
443 cols = ["t_s", "x_km", "y_km", "z_km", "vx_km_s", "vy_km_s", "vz_km_s", "mu", "J2", "J3"]
444 data = np.column_stack((sol.t, sol.y.T))

```

```

445 df = pd.DataFrame(data, columns=cols)
446
447 # write CSV
448 df.to_csv("prob2c_traj_pert.csv", index=False)
449
450
451
452
453
454
455 X_ref = truth[["x_km", "y_km", "z_km", "vx_km_s", "vy_km_s", "vz_km_s"]].to_numpy(dtype=float)
456 X_pert_traj = sol.y[:6, :].T
457
458 dx_nonlin = X_pert_traj - X_ref
459
460
461
462 ##### Pert Linearized State #####
463
464 remove = np.array([6, 8], dtype=int)
465 Phi0 = np.eye(7)
466 dx0 = np.array([x, y, z, xdot, ydot, zdot], dtype=float)
467 dx0_7 = np.hstack((dx0, 0.0))
468 y0 = np.hstack((X0_pure, Phi0.flatten()))
469
470 fun_stm = lambda t, y: stm(t, state9=y[:9], phi=y[9:].reshape(7, 7),
    rows_col_to_remove=remove, Re=Re, j2=True, j3=False)
471 sol_stm = sp.integrate.solve_ivp(fun_stm, (t_truth[0], t_truth[-1]), y0, t_eval=t_truth,
    rtol=1e-10, atol=1e-10, method="DOP853")
472
473 Phi_hist = sol_stm.y[9:, :].T.reshape(-1, 7, 7)
474
475 N = len(t_truth)
476 dx_stm_7 = np.zeros((N, 7))
477
478 for i in range(N):
479     Phi_i = Phi_hist[i, :, :]
480     dx_stm_7[i, :] = Phi_i @ dx0_7
481
482 dx_stm = dx_stm_7[:, :6]
483
484
485
486 # ----- Save delta-x histories to CSV -----
487 cols_dx = ["t_s", "dx_km", "dy_km", "dz_km", "dvx_km_s", "dvy_km_s", "dvz_km_s"]
488
489 # Nonlinear (truth) delta-x
490 df_dx_nonlin = pd.DataFrame(
491     np.column_stack((t_truth, dx_nonlin)),
492     columns=cols_dx

```

```

493 )
494 df_dx_nonlin.to_csv("prob2c_dx_nonlin.csv", index=False)
495
496 # STM-predicted delta-x
497 df_dx_stm = pd.DataFrame(
498     np.column_stack((t_truth, dx_stm)),
499     columns=cols_dx
500 )
501 df_dx_stm.to_csv("prob2c_dx_stm.csv", index=False)
502
503
504 # print("X0_pure:", X0_pure)
505 # print("X_pert :", X_pert)
506 # print("X0      :", X0)
507 # print("mu,J2,J3 in X0:", X0[6], X0[7], X0[8])
508
509
510
511
512 # #~~~~~ Plotting ~~~~~#
513
514 labels = ["δx (km)", "δy (km)", "δz (km)", "δvx (km/s)", "δvy (km/s)", "δvz (km/s)"]
515
516 fig, axs = plt.subplots(3, 2, figsize=(12, 9), sharex=True)
517 axs = axs.ravel()
518
519 for k in range(6):
520     axs[k].scatter(t_truth, dx_nonlin[:, k], label="Nonlinear truth", s=2, alpha=0.7)
521     axs[k].scatter(t_truth, dx_stm[:, k], label="STM prediction", s=2, alpha=0.5)
522     axs[k].set_ylabel(labels[k])
523     axs[k].grid(True)
524
525 axs[4].set_xlabel("t (s)")
526 axs[5].set_xlabel("t (s)")
527
528 axs[0].legend(loc="best")
529 fig.suptitle("2c: Deviation Components vs Time (Nonlinear vs STM)")
530 fig.tight_layout()
531
532
533
534 ## d
535 d_dx = dx_nonlin - dx_stm
536
537 labels_d = ["Δδx (km)", "Δδy (km)", "Δδz (km)", "Δδvx (km/s)", "Δδvy (km/s)", "Δδvz (km/s)"]
538
539 fig, axs = plt.subplots(3, 2, figsize=(12, 9), sharex=True)
540 axs = axs.ravel()
541
542 for k in range(6):

```



```
543     axs[k].scatter(t_truth, d_dx[:, k], s=2, alpha=0.7)
544     axs[k].set_ylabel(labels_d[k])
545     axs[k].grid(True)
546
547 axs[4].set_xlabel("t (s)")
548 axs[5].set_xlabel("t (s)")
549
550 fig.suptitle("2d:  $\Delta\delta x(t) = \delta x_{\text{nonlin}}(t) - \delta x_{\text{STM}}(t)$ ")
551 fig.tight_layout()
552 plt.show()
553
554
555
556
557 ## Prob 3b
558
559 import numpy as np
560 import sys
561 import json
562 import scipy as sp
563 import pandas as pd
564
565 sys.path.append("../..")
566
567
568 import numpy as np
569
570 def H_range_rangerate(R, V, Rs, Vs, eps=1e-12):
571     """
572     Measurement partials for simplified range and range-rate:
573     rho = ||R - Rs||
574     rhod = (R - Rs)·(V - Vs) / rho
575
576     Inputs: R,V,Rs,Vs are length-3 arrays (or 3x1 vectors)
577     Output: H is 2x6 Jacobian wrt X = [R; V]
578            rows: [rho, rhod], cols: [R (3), V (3)]
579     """
580     R = np.asarray(R, dtype=float).reshape(3,)
581     V = np.asarray(V, dtype=float).reshape(3,)
582     Rs = np.asarray(Rs, dtype=float).reshape(3,)
583     Vs = np.asarray(Vs, dtype=float).reshape(3,)
584
585     r = R - Rs
586     v = V - Vs
587
588     rho = np.linalg.norm(r)
589     rho_hat = r / rho
590     rhod = np.dot(r, v) / rho
591
592     # Partial
```

```

593     drho_dR = rho_hat.reshape(1, 3)      # 1x3
594     drho_dV = np.zeros((1, 3))          # 1x3
595
596     # d(rhod)/dR
597     drhod_dR = (v / rho - (np.dot(r, v) / rho**3) * r).reshape(1, 3)
598
599     # d(rhod)/dV
600     drhod_dV = rho_hat.reshape(1, 3)
601
602     H = np.block([
603         [drho_dR, drho_dV],
604         [drhod_dR, drhod_dV]]) # 2x6
605
606     return H
607
608
609
610 with open("Solution_Checks/prob3b_solution.json", "r") as validation_file:
611     my_json_obj = json.load(validation_file)
612
613     r = np.array(my_json_obj["inputs"]["spacecraft_state"]["r"], dtype=np.float64)
614     v = np.array(my_json_obj["inputs"]["spacecraft_state"]["v"], dtype=np.float64)
615     Rs = np.array(my_json_obj["inputs"]["station_state"]["Rs"], dtype=np.float64)
616     Vs = np.array(my_json_obj["inputs"]["station_state"]["Vs"], dtype=np.float64)
617     H_expected = np.array(my_json_obj["outputs"]["Htilde"]["values"], dtype=np.float64)
618
619 ##### CHECK #####
620
621 H_out1 = H_range_rangerate(r, v, Rs, Vs)
622
623 diff1 = H_out1 - H_expected
624
625 print("\n===== 3b =====")
626 print("H_out1:\n", H_out1)
627 print("\nH_expected:\n", H_expected)
628 print("\nMax abs diff:", np.max(np.abs(diff1)))
629 print("All close?", np.allclose(H_out1, H_expected, rtol=1e-12, atol=1e-12))
630
631
632
633 ##### C #####
634
635
636 def Htilde_obs_rho_rhod(r_sc, v_sc, Rs, Vs, eps=1e-12):
637     """
638     Returns 2x3 partials of [rho; rhodot] w.r.t. station position Rs.
639
640     Uses the homework's implicit ground-station kinematics:
641         Vs = omega x Rs    (omega along +z)
642     so d(Vs)/d(Rs) contributes to d(rhodot)/d(Rs).

```

```

643
644     rho  = ||r_sc - Rs||
645     rhod = ( (r_sc - Rs)·(v_sc - Vs) ) / rho
646     """
647     r_sc = np.asarray(r_sc, dtype=float).reshape(3,)
648     v_sc = np.asarray(v_sc, dtype=float).reshape(3,)
649     Rs   = np.asarray(Rs,   dtype=float).reshape(3,)
650     Vs   = np.asarray(Vs,   dtype=float).reshape(3,)
651
652     r = r_sc - Rs
653     rho = np.linalg.norm(r)
654     if rho < eps:
655         raise ValueError("Range rho is ~0; LOS undefined.")
656
657     rho_hat = r / rho
658
659     # relative velocity
660     v = v_sc - Vs
661     rv = np.dot(r, v)
662
663     # --- infer omega_z from Vs = omega x Rs with omega = [0,0,omega_z] ---
664     # Vs = [-omega_z*Rs_y, omega_z*Rs_x, 0]
665     denom = Rs[0]**2 + Rs[1]**2
666     if denom < eps:
667         raise ValueError("Cannot infer omega_z when Rs_x and Rs_y are ~0.")
668
669     omega_z = (Rs[0]*Vs[1] - Rs[1]*Vs[0]) / denom
670     omega = np.array([0.0, 0.0, omega_z])
671
672     # d(rho)/d(Rs) = -r^T/rho = -rho_hat^T
673     drho_dRs = (-rho_hat).reshape(1, 3)
674
675     # Base term (holding Vs fixed): - d(rhod)/dR
676     base = -(v / rho - (rv / rho**3) * r)
677
678     # Extra chain-rule term due to Vs(Rs): (omega x r)/rho
679     extra = np.cross(omega, r) / rho
680
681     drhod_dRs = (base + extra).reshape(1, 3)
682
683     return np.vstack((drho_dRs, drhod_dRs))
684
685
686
687 with open("Solution_Checks/prob3d_solution.json", "r") as validation_file:
688     my_json_obj = json.load(validation_file)
689
690     r = np.array(my_json_obj["inputs"]["spacecraft_state"]["r"], dtype=np.float64)
691     v = np.array(my_json_obj["inputs"]["spacecraft_state"]["v"], dtype=np.float64)
692     Rs = np.array(my_json_obj["inputs"]["station_state"]["Rs"], dtype=np.float64)

```

```
693     Vs = np.array(my_json_obj["inputs"]["station_state"]["Vs"], dtype=np.float64)
694     H_expected = np.array(my_json_obj["outputs"]["Htilde"]["values"], dtype=np.float64)
695
696 H_out = Htilde_obs_rho_rhod(r, v, Rs, Vs)
697
698 print("H_out:\n", H_out)
699 print("\nH_expected:\n", H_expected)
700
701 diff = H_out - H_expected
702 print("\nMax abs diff:", np.max(np.abs(diff)))
703 print("All close?", np.allclose(H_out, H_expected, rtol=1e-12, atol=1e-12))
704
705
706 ## Prob 4a
707
708
709 import numpy as np
710 import sys
711 import json
712 import scipy as sp
713 import pandas as pd
714 import matplotlib.pyplot as plt
715 sys.path.append("../..")
716
717 from src.Functions.stations import Stations
718
719 truth = pd.read_csv("prob2a_traj.csv")
720 t = truth["t_s"].to_numpy(float)
721 r = truth[["x_km", "y_km", "z_km"]].to_numpy(float)
722 v = truth[["vx_km_s", "vy_km_s", "vz_km_s"]].to_numpy(float)
723
724 # Stations (calling class)
725 stations = [
726     Stations("Station 1", lat_deg=-35.398333, lon_deg=148.981944),
727     Stations("Station 2", lat_deg=40.427222, lon_deg=355.749444),
728     Stations("Station 3", lat_deg=35.247164, lon_deg=243.205000),
729 ]
730
731 # Collect all measurements in one list
732 all_meas = []
733
734 for k in range(len(t)):
735     for st in stations:
736         m = st.measure(r[k], v[k], float(t[k]))
737         if m is not None:
738             all_meas.append(m)
739
740 # First/last measurement times
741
742 t_first = min(m["t"] for m in all_meas)
```

```
743 t_last = max(m["t"] for m in all_meas)
744 print("First measurement time (s):", t_first)
745 print("Last measurement time (s):", t_last)
746
747
748
749
750
751
752 # Plot by station: Range
753 plt.figure()
754 for st in stations:
755     st_meas = [m for m in all_meas if m["station"] == st.name]
756     if not st_meas:
757         continue
758     tt = np.array([m["t"] for m in st_meas], dtype=float)
759     rho = np.array([m["rho_km"] for m in st_meas], dtype=float)
760     plt.scatter(tt, rho, s=6, alpha=0.8, label=st.name)
761
762 plt.xlabel("t (s)")
763 plt.ylabel("Range  $\rho$  (km)")
764 plt.title("Range Measurements")
765 plt.grid(True)
766 plt.legend()
767
768
769 # Plot by station: Range-rate
770 plt.figure()
771 for st in stations:
772     st_meas = [m for m in all_meas if m["station"] == st.name]
773     if not st_meas:
774         continue
775     tt = np.array([m["t"] for m in st_meas], dtype=float)
776     rhod = np.array([m["rho_dot_km_s"] for m in st_meas], dtype=float)
777     plt.scatter(tt, rhod, s=6, alpha=0.8, label=st.name)
778
779 plt.xlabel("t (s)")
780 plt.ylabel("Range-rate  $\rho'$  (km/s)")
781 plt.title("Range-rate measurements")
782 plt.grid(True)
783 plt.legend()
784
785
786 # Plot by station: Elevation
787 plt.figure()
788 for st in stations:
789     st_meas = [m for m in all_meas if m["station"] == st.name]
790     if not st_meas:
791         continue
792     tt = np.array([m["t"] for m in st_meas], dtype=float)
```

```
793     elev = np.array([np.rad2deg(m["elev_rad"]) for m in st_meas], dtype=float)
794     plt.scatter(tt, elev, s=6, alpha=0.8, label=st.name)
795
796 plt.xlabel("t (s)")
797 plt.ylabel("Elevation (deg)")
798 plt.title("Elevation Angles at Measurement Times")
799 plt.grid(True)
800 plt.legend()
801
802 plt.show()
803
804
805
806
807
808 ##### D #####
809
810 def add_gaussian_noise(y: np.ndarray, sigma: float, seed: int | None = None) -> np.ndarray:
811     rng = np.random.default_rng(seed)
812     return y + rng.normal(loc=0.0, scale=sigma, size=y.shape)
813
814
815 sigma_km_s = 5e-7 # 0.5 mm/s in km/s
816
817 # Original vs noisy
818 plt.figure()
819 for st in stations:
820     st_meas = [m for m in all_meas if m["station"] == st.name]
821     if not st_meas:
822         continue
823
824     tt = np.array([m["t"] for m in st_meas], dtype=float)
825     rhod = np.array([m["rho_dot_km_s"] for m in st_meas], dtype=float)
826     rhod_noisy = add_gaussian_noise(rhod, sigma_km_s, seed=123)
827
828     plt.scatter(tt, rhod, s=6, alpha=0.8, label=f"{st.name} original")
829     plt.scatter(tt, rhod_noisy, s=6, alpha=0.6, label=f"{st.name} noisy")
830
831 plt.xlabel("t (s)")
832 plt.ylabel("Range-rate  $\dot{\rho}$  (km/s)")
833 plt.title("Range-rate: Original vs Noisy ( $\sigma = 0.5$  mm/s)")
834 plt.grid(True)
835 plt.legend()
836
837
838 # Residual plot
839 plt.figure()
840 for st in stations:
841     st_meas = [m for m in all_meas if m["station"] == st.name]
842     if not st_meas:
```



```

843         continue
844
845         tt = np.array([m["t"] for m in st_meas], dtype=float)
846         rhod = np.array([m["rho_dot_km_s"] for m in st_meas], dtype=float)
847         rhod_noisy = add_gaussian_noise(rhod, sigma_km_s, seed=123)
848         residual = rhod_noisy - rhod
849
850         plt.scatter(tt, residual, s=6, alpha=0.8, label=st.name)
851
852     plt.xlabel("t (s)")
853     plt.ylabel("Noisy - original (km/s)")
854     plt.title("Range-Rate Noise Residuals")
855     plt.grid(True)
856     plt.legend()
857
858     plt.show()
859
860
861 # Prob 4c
862
863 import numpy as np
864 import sys
865 import json
866 import scipy as sp
867 import pandas as pd
868 import matplotlib.pyplot as plt
869 sys.path.append("../..")
870
871 from src.Functions.stations import Stations
872
873
874
875
876
877 def RUdoppler( station: Stations, r_sc_eci: np.ndarray, v_sc_eci: np.ndarray, t: float,
878               f_tr_ref_hz: float = 8.44e9, c_km_s: float = 2.99792458e5) -> dict:
879     elev = station.elevation(t, r_sc_eci)
880
881     if elev < station.elevation_mask_rad:
882         return {"station": station.name, "t": t, "RU": np.nan, "f_shift_hz": np.nan,
883               "elev_rad": elev}
884
885     m = station.measure(r_sc_eci, v_sc_eci, t)
886     rho_km = float(m["rho_km"])
887     rhodot_km_s = float(m["rho_dot_km_s"])
888
889     f_shift = -2.0 * rhodot_km_s * f_tr_ref_hz / c_km_s
890     RU = (221.0 / 749.0) * (rho_km / c_km_s) * f_tr_ref_hz
891
892     return {"station": station.name, "t": t, "RU": RU, "f_shift_hz": f_shift, "elev_rad":
893           elev}

```

```

891
892
893
894 truth = pd.read_csv("prob2a_traj.csv")
895 t = truth["t_s"].to_numpy(float)
896 r = truth[["x_km", "y_km", "z_km"]].to_numpy(float)
897 v = truth[["vx_km_s", "vy_km_s", "vz_km_s"]].to_numpy(float)
898
899 stations = [
900     Stations("Station 1", lat_deg=-35.398333, lon_deg=148.981944),
901     Stations("Station 2", lat_deg=40.427222, lon_deg=355.749444),
902     Stations("Station 3", lat_deg=35.247164, lon_deg=243.205000),
903 ]
904
905 all_dsn = []
906
907 for k in range(len(t)):
908     for st in stations:
909         y = RUdoppler(st, r[k], v[k], float(t[k])) # returns dict
910         all_dsn.append(y)
911
912
913 plt.figure()
914 for st in stations:
915     st_dsn = [d for d in all_dsn if d["station"] == st.name and np.isfinite(d["RU"])]
916     if not st_dsn:
917         continue
918     plt.plot([d["t"] for d in st_dsn],
919             [d["RU"] for d in st_dsn],
920             marker=".", linestyle="None", label=st.name)
921
922 plt.xlabel("t (s)")
923 plt.ylabel("Range Units (RU)")
924 plt.title("2-way Range in Range Units (RU)")
925 plt.grid(True)
926 plt.legend()
927
928 plt.figure()
929 for st in stations:
930     st_dsn = [d for d in all_dsn if d["station"] == st.name and np.isfinite(d["f_shift_hz"])]
931     if not st_dsn:
932         continue
933     plt.plot([d["t"] for d in st_dsn],
934             [d["f_shift_hz"] for d in st_dsn],
935             marker=".", linestyle="None", label=st.name)
936
937 plt.xlabel("t (s)")
938 plt.ylabel("Doppler shift (Hz)")
939 plt.title("2-way Doppler shift (Hz)")
940 plt.grid(True)

```

```
941 plt.legend()
942 plt.show()
943
944
945 ##Prob 4d
946
947 import numpy as np
948 import sys
949 import json
950 import scipy as sp
951 import pandas as pd
952 import matplotlib.pyplot as plt
953 sys.path.append("../..")
954
955 from src.Functions.stations import Stations
956
957 truth = pd.read_csv("prob2a_traj.csv")
958 t = truth["t_s"].to_numpy(float)
959 r = truth[["x_km", "y_km", "z_km"]].to_numpy(float)
960 v = truth[["vx_km_s", "vy_km_s", "vz_km_s"]].to_numpy(float)
961
962 # Stations (calling class)
963 stations = [
964     Stations("Station 1", lat_deg=-35.398333, lon_deg=148.981944),
965     Stations("Station 2", lat_deg=40.427222, lon_deg=355.749444),
966     Stations("Station 3", lat_deg=35.247164, lon_deg=243.205000),
967 ]
968
969
970
971
972
973
974
```

src\Functions\stations.py

```

1  import numpy as np
2
3
4  class Stations:
5      def __init__(
6          self,
7          name: str,
8          lat_deg: float,
9          lon_deg: float,
10         elevation_mask_deg: float = 10,
11         radius_earth: float = 6378,
12         theta0_deg: float = 122,
13         w_earth_rad_per_s: float = 2*np.pi/(24*60*60)
14     ):
15
16         self.name = name
17         self.lat_deg = float(lat_deg)
18         self.lon_deg = float(lon_deg)
19         self.elevation_mask_deg = float(elevation_mask_deg)
20         self.radius_earth = float(radius_earth)
21         self.theta0_deg = float(theta0_deg)
22         self.w_earth_rad_per_s = float(w_earth_rad_per_s)
23         self.deg2radians()
24         self.r_ecef = self.lat_lon2ecef()
25
26
27
28     def deg2radians(self) -> None:
29         self.lat_rad:float = np.deg2rad(self.lat_deg)
30         self.lon_rad:float = np.deg2rad(self.lon_deg)
31         self.elevation_mask_rad: float = np.deg2rad(self.elevation_mask_deg)
32         self.theta0_rad:float = np.deg2rad(self.theta0_deg)
33
34
35
36     def lat_lon2ecef(self) -> np.ndarray:
37         .....
38         x = self.radius_earth * np.cos(self.lat_rad) * np.cos(self.lon_rad)
39         y = self.radius_earth * np.cos(self.lat_rad) * np.sin(self.lon_rad)
40         z = self.radius_earth * np.sin(self.lat_rad)
41         return np.array([x, y, z], dtype=float)
42
43
44     def ecef2eci(self, t: float, r_sc_ecef: np.ndarray, v_sc_ecef: np.ndarray) ->
tuple[np.ndarray, np.ndarray, np.ndarray]:
45         theta = self.theta0_rad + self.w_earth_rad_per_s*t
46         cos: float = np.cos(theta)
47         sin: float = np.sin(theta)

```

```
48
49     R3: np.ndarray = np.array([[cos, -sin, 0],
50                               [sin, cos, 0],
51                               [ 0,   0,   1]])
52
53
54     r_eci = R3 @ r_sc_ecef
55     v_eci = R3 @ v_sc_ecef + np.cross(np.array([0.0, 0.0, self.w_earth_rad_per_s]),
r_eci)
56     X_eci = np.hstack([r_eci, v_eci])
57
58     return r_eci, v_eci, X_eci
59
60     def elevation(self, t: float, r_sc_eci: np.ndarray) -> float:
61
62         v_zero = np.zeros(3)
63         r_st, _, _ = self.ecef2eci(t, self.r_ecef, v_zero)
64         rho_vec: np.ndarray = r_sc_eci - r_st
65         rho_hat: np.ndarray = rho_vec / np.linalg.norm(rho_vec)
66         up: np.ndarray = r_st / np.linalg.norm(r_st)
67         elev = np.arcsin(np.dot(rho_hat, up))
68         return float(elev)
69
70
71     def measure(self, r_sc_eci: np.ndarray, v_sc_eci: np.ndarray, t: float) -> dict | None:
72
73         elev = self.elevation(t, r_sc_eci)
74
75         if elev < self.elevation_mask_rad:
76             return None
77
78         v_zero = np.zeros(3)
79         r_st, v_st, _ = self.ecef2eci(t, self.r_ecef, v_zero)
80
81         rho_vec = r_sc_eci - r_st
82         rho = np.linalg.norm(rho_vec)
83         rho_hat = rho_vec / rho
84
85         rho_dot = np.dot(rho_hat, (v_sc_eci - v_st))
86
87         return {
88             "station": self.name,
89             "t": t,
90             "rho_km": rho,
91             "rho_dot_km_s": rho_dot,
92             "elev_rad": elev
93         }
94
95
96
```

97
98
99
100
101
102
103
104
105
106
107
108
109
110
111